

Solution Requirements

Date	03 July 2026
Team ID	N/A
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Functional Requirements (FR) define specific behaviors/features of OptiCrop. Non-Functional Requirements (NFR) define how well the system performs.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	No login is required; OptiCrop is a public tool — any farmer, researcher, or visitor can access the Home, About, and Find Your Crop pages directly.	Low
2	Authorization levels	A single public user role exists in the current version. A future admin role could manage dataset uploads and retraining.	Low
3	External interfaces	None required currently — the app is self-contained using the trained model (model.pkl) and scaler (scaler.pkl). Future scope: a weather API for live rainfall/temperature input.	Low
4	Core prediction processing	The system must accept N, P, K, temperature, humidity, pH, and rainfall values from the form, scale them, and return a predicted crop using the trained model.	High
5	Reporting	The predicted crop is displayed instantly on the Find Your Crop page. Future scope: downloadable prediction summary (PDF/CSV).	Medium
6	Business rules	The system may only recommend a crop from the 22 crop classes present in the training dataset; all numeric inputs must be validated before prediction.	High
7	Compliance to laws or regulations	No personal or sensitive data is collected or stored, so no specific data-privacy regulation (e.g., GDPR) currently applies.	Low
8	Other	Support periodic retraining of the model (train_model.py) as new agricultural data becomes available.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Since the model and scaler are pre-loaded in memory, predictions must return almost instantly after form submission.	Prediction response in < 1 second
2	Scalability	The Flask app should be deployable behind a production WSGI server to handle more concurrent users than the built-in dev server.	Designed to extend via Gunicorn/WSGI + load balancing
3	Security & Data Privacy	No personal data is stored; soil/climate inputs are processed transiently per request only.	No persistent storage of user-submitted input
4	Reliability & Availability	The application should run consistently without crashing on invalid or missing form input.	Target 99% uptime once deployed beyond local testing
5	Usability & Accessibility	The interface uses simple labels (N, P, K, temperature, humidity, pH, rainfall) and a clean layout usable by non-technical farmers.	Usable without prior technical training
6	Other (Maintainability)	Code is organized into clear modules (dataset generation, training script, Flask app, templates) so the model can be retrained or the UI updated independently.	New model can be swapped in by replacing model.pkl / scaler.pkl